

GRUPPO 31

PSS

Assignment 3 - Clean Code

Contents

1	Applicazione scelta	2
2	Analisi di EventAdapter	3
2.1	Codice	3
2.2	Analisi del codice	7
2.2.1	Nomi	7
2.2.2	Commenti	7
2.2.3	Formattazione	8
2.2.4	Funzioni	8
2.3	Codice ristrutturato	8
2.4	Analisi del codice ristrutturato	12
3	Classe JsonParser.java	13
3.1	Ruolo	13
3.2	Codice originale	13
3.3	Analisi di Clean Code	15
3.3.1	Nomi	15
3.3.2	Responsabilità	16
3.3.3	Commenti	16
3.3.4	Formattazione	16
3.3.5	Hardcoding	16
3.3.6	Gestione degli errori	17
3.4	Code Refactoring	17
3.5	Modifiche effettuate	19
4	Classe User.java	20
4.1	Ruolo	20
4.2	Codice originale	20
4.3	Analisi di Clean Code	22
4.3.1	Nomi	22
4.3.2	Responsabilità	22
4.3.3	Formattazione	23
4.3.4	Hardcoding	23
4.3.5	Commenti	23
4.3.6	Errori logici	23
4.4	Code Refactoring	23
4.5	Modifiche effettuate	25

1 Applicazione scelta

2 Analisi di EventAdapter

2.1 Codice

```
1 package it.marcosoft.ticketwave.adapter;
2
3 import android.annotation.SuppressLint;
4 import android.content.Context;
5 import android.database.Cursor;
6 import android.database.sqlite.SQLiteDatabase;
7 import android.graphics.drawable.AnimatedVectorDrawable;
8 import android.graphics.drawable.Drawable;
9 import android.util.Log;
10 import android.view.GestureDetector;
11 import android.view.LayoutInflater;
12 import android.view.MotionEvent;
13 import android.view.View;
14 import android.view.ViewGroup;
15 import android.widget.ImageView;
16 import android.widget.LinearLayout;
17 import android.widget.TextView;
18 import android.widget.Toast;
19
20 import androidx.annotation.NonNull;
21 import androidx.recyclerview.widget.RecyclerView;
22 import androidx.vectordrawable.graphics.drawable.AnimatedVectorDrawableCompat;
23
24 import com.squareup.picasso.Picasso;
25
26 import java.util.List;
27
28 import it.marcosoft.ticketwave.EventModel.Event;
29 import it.marcosoft.ticketwave.R;
30 import it.marcosoft.ticketwave.data.LikedData;
31 import it.marcosoft.ticketwave.util.db.DBHelperLiked;
32
33 public class EventAdapter extends RecyclerView.Adapter<EventAdapter.ViewHolder> {
34
35     // VARIABILI PER ANIMAZIONE LIKE
36     ImageView heart;
37     ImageView cover;
38     AnimatedVectorDrawableCompat avd;
39     AnimatedVectorDrawable avd2;
40
41     // VARIABILI PER ANIMAZIONE DISLIKE
42     ImageView disheart;
43     AnimatedVectorDrawableCompat avd3;
44     AnimatedVectorDrawable avd4;
45     private final LayoutInflater inflater;
46     private List<Event> eventList;
47
48     public EventAdapter(Context context, List<Event> eventList) {
49         this.inflater = LayoutInflater.from(context);
50         this.eventList = eventList;
51     }
52
53
54     @NonNull
```

```

55  @Override
56  public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
57      View view = LayoutInflater.inflate(R.layout.event_list_item_card, parent,
58          ↪ false);
59      return new ViewHolder(view);
60
61  @SuppressWarnings("ClickableViewAccessibility")
62  @Override
63  public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
64      Event event = eventList.get(position);
65      // Retrieve data from the event object
66      String title = event.getName();
67      String location = event.getVenue().getName();
68      String date = event.getDate();
69      String description = event.getClassifications().get(0).toStringPretty();
70      String imgUrl = event.getImages().get(0).getUrlImage();
71      String id = event.getId();
72
73
74      // Bind data to the ViewHolder
75      holder.textTitle.setText(title);
76      holder.textLocation.setText(location);
77      holder.textDate.setText(date);
78      holder.textDescription.setText(description);
79      holder.tagCard.setTag(id);
80      Picasso.get().load(imgUrl).into(holder.imageView);
81
82      // Set the click listener for the card view
83      GestureDetector gestureDetector = new
84          ↪ GestureDetector(holder.itemView.getContext(),
85              new GestureDetector.SimpleOnGestureListener() {
86                  @Override
87                  public boolean onSingleTapConfirmed(@NonNull MotionEvent e) {
88                      // Handle single tap (optional)
89                      return super.onSingleTapConfirmed(e);
90                  }
91
92                  @Override
93                  public boolean onDoubleTap(@NonNull MotionEvent e) {
94
95                      //PREPARAZIONE PER ANIMAZIONE LIKE/DISLIKE
96                      cover = holder.itemView.findViewById(R.id.event_image);
97                      heart = holder.itemView.findViewById(R.id.like_animation);
98                      disheart =
99                      ↪ holder.itemView.findViewById(R.id.dislike_animation);
100                      final Drawable drawable=heart.getDrawable();
101                      final Drawable drawable2=disheart.getDrawable();
102
103                      String idEvent = String.valueOf(holder.tagCard.getTag());
104                      String userId = "userId"; // Sostituisci "userId" con l'id
105                      ↪ dell'utente reale
106
107                      // Verifica se l'evento è già presente nel database
108                      DBHelperLiked dbHelper = new
109                      ↪ DBHelperLiked(holder.itemView.getContext());
110                      SQLiteDatabase db = dbHelper.getReadableDatabase();

```

```

108
109 String selection = DBHelperLiked.COLUMN_EVENT_ID + " = ? AND "
    ↳ + DBHelperLiked.COLUMN_USER_ID + " = ?";
110 String[] selectionArgs = {idEvent, userId};
111
112 Cursor cursor = db.query(
113     DBHelperLiked.TABLE_LIKED_EVENTS,
114     null,
115     selection,
116     selectionArgs,
117     null,
118     null,
119     null
120 );
121
122 boolean isEventAlreadyLiked = cursor.getCount() > 0;
123 cursor.close();
124 db.close();
125
126 // Aggiungi l'evento al database solo se non è già presente
127 if (!isEventAlreadyLiked) {
128     dbHelper = new
129     ↳ DBHelperLiked(holder.itemView.getContext());
130     db = dbHelper.getWritableDatabase();
131
132     //ANIMAZIONE DEL LIKE
133     if (heart != null) {
134         heart.setAlpha(1f);
135     }
136     if (drawable instanceof AnimatedVectorDrawableCompat){
137         avd = (AnimatedVectorDrawableCompat) drawable;
138         avd.start();
139     } else if (drawable instanceof AnimatedVectorDrawable) {
140         avd2=(AnimatedVectorDrawable) drawable;
141         avd2.start();
142     }
143
144     // Utilizza la nuova classe LikedData estesa
145     LikedData likedData = new LikedData(
146         idEvent,
147         userId,
148         title,
149         location,
150         date,
151         description,
152         imgUrl
153     );
154
155     dbHelper.addLikedEvent(likedData);
156     db.close();
157     Toast.makeText(holder.itemView.getContext(), "Liked
158     ↳ event!", Toast.LENGTH_SHORT).show();
159
160 } else {
161     // Evento già presente nel database
162     dbHelper = new
163     ↳ DBHelperLiked(holder.itemView.getContext());
164     db = dbHelper.getWritableDatabase();

```

```

162
163         // Utilizza la nuova classe LikedData estesa
164         LikedData likedData = new LikedData(
165             idEvent,
166             userId,
167             title,
168             location,
169             date,
170             description,
171             imgUrl
172         );
173         dbHelper.removeLikedEvent(likedData.getEventId());
174
175
176         //ANIMAZIONE DEL DISLIKE
177         if (disheart != null) {
178             disheart.setAlpha(1f);
179         }
180         if (drawable2 instanceof AnimatedVectorDrawableCompat){
181             avd3 = (AnimatedVectorDrawableCompat) drawable2;
182             avd3.start();
183         } else if (drawable2 instanceof AnimatedVectorDrawable) {
184             avd4=(AnimatedVectorDrawable) drawable2;
185             avd4.start();
186         }
187
188         Toast.makeText(holder.itemView.getContext(), "Disliked
189             ↪ event!", Toast.LENGTH_SHORT).show();
190
191         return true;
192     }
193 }
194
195 holder.itemView.setOnTouchListener((v, eventCard) ->
196     ↪ gestureDetector.onTouchEvent(eventCard));
197
198
199 @Override
200 public int getItemCount() {
201     return eventList.size();
202 }
203
204 public static class ViewHolder extends RecyclerView.ViewHolder {
205
206     final TextView textTitle;
207     final TextView textLocation;
208     final TextView textDate;
209     final TextView textDescription;
210     final ImageView imageView;
211     final LinearLayout tagCard;
212
213     public ViewHolder(@NonNull View itemView) {
214         super(itemView);
215         textTitle = itemView.findViewById(R.id.event_name);
216         textLocation = itemView.findViewById(R.id.event_location);
217         textDate = itemView.findViewById(R.id.event_date);

```

```

218         imageView = itemView.findViewById(R.id.event_image);
219         textDescription = itemView.findViewById(R.id.event_description);
220         tagCard = itemView.findViewById(R.id.cardId);
221     }
222 }
223
224
225 @SuppressWarnings("NotifyDataSetChanged")
226 public void setEventList(List<Event> events) {
227     this.eventList = events;
228     notifyDataSetChanged();
229 }
230 }

```

2.2 Analisi del codice

2.2.1 Nomi

All'interno del codice sono presenti delle variabili di classe che possono essere dichiarate direttamente all'interno delle funzioni, una volta riscritte correttamente. Inoltre, molte di esse presentano un nome che introduce ambiguità, perchè poco esplicativo. L'esempio più lampante lo forniscono:

- avd1
- avd2
- avd3
- avd4

A dimostrazione dell'ambiguità dei nomi troviamo i commenti `//VARIABILI PER ANIMAZIONE LIKE` e `//VARIABILI PER ANIMAZIONE DISLIKE`, scritti per spiegare per cosa verranno utilizzate le variabili. I nomi delle funzioni, invece, risultano essere corretti, in quanto sono tutti degli `@Override` di funzioni già esistenti.

2.2.2 Commenti

Nel codice originale non sono presenti commenti a parti di codice vecchie o inutilizzate. Sono però presenti svariati commenti a parti di codice che dovrebbero essere delle funzioni a se stanti e che diventerebbero autoesplicative tramite il nome:

- `// Retrieve data from the event object`
- `// Bind data to the ViewHolder`
- `// Set the click listener for the card view`
- `// PREPARAZIONE PER ANIMAZIONE LIKE/DISLIKE`
- `// Verifica se l'evento è già presente nel database`
- `// ANIMAZIONE DEL LIKE`
- `// ANIMAZIONE DEL DISLIKE`

Lo stesso si verifica con le variabili di classe, che possedendo dei nomi poco adeguati, vengono introdotte con i commenti:

- `// VARIABILI PER ANIMAZIONE LIKE`
- `// VARIABILI PER ANIMAZIONE DISLIKE`

Altri commenti risultano invece superflui:

- `// Handle single tap (optional)`

- // Sostituisci "userId" con l'id dell'utente reale
- // Aggiungi l'evento al database solo se non è già presente
- // Utilizza la nuova classe LikedData estesa
- // Evento già presente nel database

2.2.3 Formattazione

L'ordine con cui le funzioni sono state scritte è corretto, così come l'indentazione del codice, quindi i concetti di distanza verticale e di lunghezza orizzontale vengono rispettati.

2.2.4 Funzioni

La funzione `onBindViewHolder` risulta essere eccessivamente lunga e quindi dispersiva. Molte parti di codice al suo interno dovrebbero essere funzioni esterne, dato che svolgono un compito isolabile, come per esempio:

```
// Bind data to the ViewHolder
holder.textTitle.setText(title);
holder.textLocation.setText(location);
holder.textDate.setText(date);
holder.textDescription.setText(description);
holder.tagCard.setTag(id);
Picasso.get().load(imgUrl).into(holder.imageView);
}
```

Le altre funzioni invece risultano scritte correttamente, in quanto tutte svolgono un unico compito, hanno nomi appropriati e non necessitano di un numero elevato di parametri.

2.3 Codice ristrutturato

```
1 package it.marcosoft.ticketwave.adapter;
2
3 import android.annotation.SuppressLint;
4 import android.content.Context;
5 import android.database.Cursor;
6 import android.database.sqlite.SQLiteDatabase;
7 import android.graphics.drawable.AnimatedVectorDrawable;
8 import android.graphics.drawable.Drawable;
9 import android.view.GestureDetector;
10 import android.view.LayoutInflater;
11 import android.view.MotionEvent;
12 import android.view.View;
13 import android.view.ViewGroup;
14 import android.widget.ImageView;
15 import android.widget.LinearLayout;
16 import android.widget.TextView;
17 import android.widget.Toast;
18
19 import androidx.annotation.NonNull;
20 import androidx.recyclerview.widget.RecyclerView;
21 import androidx.vectordrawable.graphics.drawable.AnimatedVectorDrawableCompat;
22
23 import com.squareup.picasso.Picasso;
24
25 import java.util.List;
26
```

```

27 import it.marcosoft.ticketwave.EventModel.Event;
28 import it.marcosoft.ticketwave.R;
29 import it.marcosoft.ticketwave.data.LikedData;
30 import it.marcosoft.ticketwave.util.db.DBHelperLiked;
31
32 public class EventAdapter extends RecyclerView.Adapter<EventAdapter.ViewHolder> {
33     private final LayoutInflater inflater;
34     private List<Event> eventList;
35
36     public EventAdapter(Context context, List<Event> eventList) {
37         this.inflater = LayoutInflater.from(context);
38         this.eventList = eventList;
39     }
40
41
42     @NonNull
43     @Override
44     public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
45         View view = inflater.inflate(R.layout.event_list_item_card, parent,
46             ↪ false);
47         return new ViewHolder(view);
48
49         @SuppressWarnings("ClickableViewAccessibility")
50         @Override
51         public void onBindViewHolder(@NonNull ViewHolder viewHolder, int position) {
52             Event event = eventList.get(position);
53
54             EventInfo eventInfo = mapEventToEventInfo(event);
55
56             bindEventInfoToViewHolder(viewHolder, eventInfo);
57
58             setupDoubleTapLikeGesture(viewHolder, eventInfo);
59         }
60
61         private EventInfo mapEventToEventInfo(Event event){
62             String description = "";
63             String imgUrl = "";
64
65             if(!event.getClassifications().isEmpty())
66                 description = event.getClassifications().get(0).toStringPretty();
67
68             if(!event.getImages().isEmpty())
69                 imgUrl = event.getImages().get(0).getUrlImage();
70
71             return new EventInfo(
72                 event.getName(),
73                 event.getVenue().getName(),
74                 event.getDate(),
75                 description,
76                 imgUrl,
77                 event.getId()
78             );
79         }
80
81         private void bindEventInfoToViewHolder(ViewHolder viewHolder, EventInfo
82             ↪ eventInfo){
83             viewHolder.textTitle.setText(eventInfo.title());

```

```

83     viewHolder.textLocation.setText(eventInfo.venueName());
84     viewHolder.textDate.setText(eventInfo.eventDate());
85     viewHolder.textDescription.setText(eventInfo.description());
86     viewHolder.tagCard.setTag(eventInfo.eventId());
87     Picasso.get().load(eventInfo.imgUrl()).into(viewHolder.imageView);
88 }
89
90 @SuppressWarnings("ClickableViewAccessibility")
91 private void setupDoubleTapLikeGesture(ViewHolder viewHolder, EventInfo
    ↪ eventInfo){
92     GestureDetector gestureDetector = new
    ↪ GestureDetector(viewHolder.itemView.getContext(),
93         new GestureDetector.SimpleOnGestureListener() {
94             @Override
95             public boolean onSingleTapConfirmed(@NonNull MotionEvent e) {
96                 return super.onSingleTapConfirmed(e);
97             }
98
99             @Override
100             public boolean onDoubleTap(@NonNull MotionEvent e) {
101                 toggleLikeStatus(viewHolder, eventInfo);
102                 return true;
103             }
104         });
105
106     viewHolder.itemView.setOnTouchListener((v, eventCard) ->
    ↪ gestureDetector.onTouchEvent(eventCard));
107 }
108
109 private void toggleLikeStatus(ViewHolder viewHolder, EventInfo eventInfo){
110     String userId = "userId";
111
112     if (isEventLiked(viewHolder.itemView.getContext(), eventInfo.eventId(),
    ↪ userId)) {
113         removeEventFromLiked(viewHolder, eventInfo, userId);
114     } else {
115         addEventToLiked(viewHolder, eventInfo, userId);
116     }
117 }
118
119 private boolean isEventLiked(Context context, String eventId, String userId) {
120     DBHelperLiked dbHelper = new DBHelperLiked(context);
121     SQLiteDatabase db = dbHelper.getReadableDatabase();
122
123     String selection = DBHelperLiked.COLUMN_EVENT_ID + " = ? AND " +
124         DBHelperLiked.COLUMN_USER_ID + " = ?";
125     String[] selectionArgs = {eventId, userId};
126
127     Cursor cursor = db.query(
128         DBHelperLiked.TABLE_LIKED_EVENTS,
129         null,
130         selection,
131         selectionArgs,
132         null,
133         null,
134         null
135     );
136

```

```

137         boolean eventLiked = cursor.getCount() > 0;
138         cursor.close();
139         db.close();
140
141         return eventLiked;
142     }
143
144     private void removeEventFromLiked(ViewHolder viewHolder, EventInfo eventInfo,
145 ↪ String userId) {
146         ImageView dislikeAnimationView =
147 ↪ viewHolder.itemView.findViewById(R.id.dislike_animation);
148         playAnimation(dislikeAnimationView);
149
150         DBHelperLiked dbHelper = new DBHelperLiked(viewHolder.itemView.getContext());
151         dbHelper.removeLikedEvent(eventInfo.eventId());
152
153         Toast.makeText(viewHolder.itemView.getContext(),
154             "Disliked event!", Toast.LENGTH_SHORT).show();
155     }
156
157     private void addEventToLiked(ViewHolder viewHolder, EventInfo eventInfo, String
158 ↪ userId) {
159         ImageView likeAnimationView =
160 ↪ viewHolder.itemView.findViewById(R.id.like_animation);
161         playAnimation(likeAnimationView);
162
163         DBHelperLiked dbHelper = new DBHelperLiked(viewHolder.itemView.getContext());
164         dbHelper.addLikedEvent(new LikedData(
165             eventInfo.eventId(),
166             userId,
167             eventInfo.title(),
168             eventInfo.venueName(),
169             eventInfo.eventDate(),
170             eventInfo.description(),
171             eventInfo.imgUrl()
172         ));
173
174         Toast.makeText(viewHolder.itemView.getContext(),
175             "Liked event!", Toast.LENGTH_SHORT).show();
176     }
177
178     private void playAnimation(ImageView animationView) {
179         Drawable drawable = animationView.getDrawable();
180
181         animationView.setAlpha(1f);
182
183         if (drawable instanceof AnimatedVectorDrawableCompat animatedDrawableCompat) {
184             animatedDrawableCompat.start();
185         } else if (drawable instanceof AnimatedVectorDrawable animatedDrawable) {
186             animatedDrawable.start();
187         }
188     }
189
190     @Override
191     public int getItemCount() {
192         return eventList.size();
193     }

```

```

191 public static class ViewHolder extends RecyclerView.ViewHolder {
192
193     final TextView textTitle;
194     final TextView textLocation;
195     final TextView textDate;
196     final TextView textDescription;
197     final ImageView imageView;
198     final LinearLayout tagCard;
199
200     public ViewHolder(@NonNull View itemView) {
201         super(itemView);
202         textTitle = itemView.findViewById(R.id.event_name);
203         textLocation = itemView.findViewById(R.id.event_location);
204         textDate = itemView.findViewById(R.id.event_date);
205         imageView = itemView.findViewById(R.id.event_image);
206         textDescription = itemView.findViewById(R.id.event_description);
207         tagCard = itemView.findViewById(R.id.cardId);
208     }
209 }
210
211 @SuppressWarnings("NotifyDataSetChanged")
212 public void setEventList(List<Event> events) {
213     this.eventList = events;
214     notifyDataSetChanged();
215 }
216
217 public record EventInfo(
218     String title,
219     String venueName,
220     String eventDate,
221     String description,
222     String imgUrl,
223     String eventId) {}
224 }
225 }

```

2.4 Analisi del codice ristrutturato

La funzione `onBindViewHolder` è stata scomposta in più funzioni:

- **mapEventToEventInfo**
Estrae i dati dall'evento e li inserisce all'interno di un record. Si è scelto di immagazzinare le informazioni all'interno di un record perchè sono molte e quindi è più corretto utilizzare una nuova struttura dedicata, piuttosto che passare tutto come parametro in una singola funzione.
- **bindEventInfoToViewHolder**
Inserisce tutti i dati che sono stati memorizzati nel record all'interno del `ViewHolder`.
- **setupDoubleTapLikeGesture**
Si occupa di generare il `GestureDetector`, che gestisce l'aggiunta e la rimozione di un item dalla lista dei preferiti. A sua volta la funzione `onDoubleTap` conteneva troppo codice che è stato portato all'esterno, alleggerendola.
 - **toggleLikeStatus**
Si occupa di chiamare le funzioni `removeLike` e `addLike`, dopo aver controllato, tramite `isEventLiked`, se l'evento è già impostato come liked o meno.
 - **isEventLiked**
Controllo effettivo dell'impostazione dell'evento. Restituisce un valore booleano che è `true` se l'evento è liked e `false` altrimenti.

- `removeEventFromLiked`
Se l'evento è liked, questa funzione lo rimuove dalla lista dei preferiti, mostrando l'animazione dell'immagine.
- `addEventToLiked`
Se l'evento è unliked, questa funzione lo aggiunge alla lista dei preferiti, preoccupandosi di associare poi l'evento all'utente che ha effettuato l'operazione. Anche in questo caso viene mostrata l'animazione dell'immagine.
- `playAnimation`
Si occupa della gestione effettiva dell'animazione.

Una volta eseguita la ristrutturazione, è stato possibile spostare le variabili di classe all'interno delle nuove funzioni, andando a rinominarle per renderle più comprensibili:

- La variabile di classe `heart` è stata spostata all'interno della funzione `addEventToLiked` ed è stata rinominata `likeAnimationView`.
- La variabile di classe `disheart` è stata spostata all'interno della funzione `removeEventFromLiked` ed è stata rinominata `dislikeAnimationView`.
- Le variabili di classe `avd` e `avd2` sono state spostate all'interno della funzione `playAnimation` e sono state rispettivamente nominate `animatedDrawableCompat` e `animatedDrawable`.
- Le variabili di classe `avd3` e `avd4` sono state rimosse perchè inutili nella nuova struttura del codice.

Sono stati rimossi i commenti superflui e quelli inutili, dato che la scomposizione del codice in numerose funzioni e la rinominazione delle variabili rendono tutto autoesplicativo.

3 Classe `JsonParser.java`

3.1 Ruolo

Nonostante il nome, la classe non si occupa esclusivamente di fare il parsing di file JSON, ma anche di gestire la comunicazione con la API esterna di TicketMaster, su cui la applicazione si basa per ottenere gli eventi futuri da mostrare agli utenti.

Questo la trasforma in una classe di tipo Client, che comunica con un servizio esterno, la API di TicketMaster in questo caso, per ottenere i dati necessari al corretto funzionamento dell'applicazione.

La funzione principale infatti si occupa di creare un URL dinamico, utilizzando l'endpoint base e aggiungendo la chiave API e eventuali parametri di query. Successivamente stabilisce e gestisce una connessione HTTP verso questo URL, ottenendo la risposta in JSON da parte di TicketMaster. Infine esegue il parsing della risposta ottenuta, estraendo gli eventi e notificando il listener dei dati ottenuti.

3.2 Codice originale

```

1 package it.marcosoft.ticketwave.NetworkActivity;
2
3 import android.content.Context;
4 import android.util.Log;
5
6 import androidx.recyclerview.widget.RecyclerView;
7
8 import com.android.volley.Request;
9 import com.android.volley.Response;
10 import com.android.volley.VolleyError;
11 import com.android.volley.toolbox.JsonObjectRequest;
12
13 import org.json.JSONArray;
14 import org.json.JSONObject;
15
16 import java.util.ArrayList;

```

```

17 import java.util.List;
18
19 import it.marcosoft.ticketwave.EventModel.Event;
20
21 /**
22  * Utility class for parsing JSON responses from the Ticketmaster API.
23  */
24 public class JsonParser {
25     // API key for accessing the Ticketmaster API
26     private static final String API_KEY = "apikey=KqtxCDlnofSteZ63m7gmezFR8PR34o78";
27
28     // Endpoint for the API request
29     private final String root;
30
31     // Query parameters for the API request
32     private final List<String> queryParams;
33
34     // List to store the parsed events
35     private final List<Event> eventsList;
36
37     // Android application context
38     private final OnEventsParsedListener onEventsParsedListener;
39
40     // RecyclerView to display the parsed events
41
42     /**
43      * Constructor for the JsonParser class.
44      *
45      * @param root          The root endpoint for the API request.
46      * @param queryParams   List of query parameters for the API request.
47      */
48     public JsonParser(String root, List<String> queryParams, OnEventsParsedListener
49         ↪ listener) {
50         this.root = root;
51         this.queryParams = queryParams;
52         this.eventsList = new ArrayList<>();
53         this.onEventsParsedListener = listener;
54     }
55
56     /**
57      * Creates a JsonObjectRequest for the API call.
58      *
59      * @return JsonObjectRequest for the API call.
60      */
61     public JsonObjectRequest jsonParse() {
62         // Build the URL for the API request
63         StringBuilder urlBuilder = new
64             ↪ StringBuilder("https://app.ticketmaster.com/").append(root).append("?");
65         for (String queryParam : queryParams) {
66             urlBuilder.append(queryParam).append("&");
67         }
68         String url = urlBuilder.append(API_KEY).toString();
69         Log.d("api", url);
70
71         // Create a JsonObjectRequest for the API call
72
73         return new JsonObjectRequest(
74             Request.Method.GET, url, null, new Response.Listener<JSONObject>() {

```

```

73
74     @Override
75     public void onResponse(JSONObject response) {
76         try {
77             // Parse the JSON response based on the API endpoint
78             if ("discovery/v2/events".equals(root)) {
79                 JSONObject embedded = response.getJSONObject("_embedded");
80                 JSONArray events = embedded.getJSONArray("events");
81                 for (int i = 0; i < events.length(); i++) {
82                     JSONObject eventObj = events.getJSONObject(i);
83                     Event event = new Event(eventObj);
84                     eventsList.add(event);
85                 }
86             } else {
87                 // Handle unknown API endpoint
88                 throw new Exception();
89             }
90         } catch (Exception e) {
91             // Handle the exception (e.g., log it)
92             Log.e("JsonParser", "Error parsing JSON", e);
93         }
94
95         if (onEventsParsedListener != null) {
96             onEventsParsedListener.onEventsParsed(eventsList);
97         }
98     }
99     }, new Response.ErrorListener() {
100         @Override
101         public void onErrorResponse(VolleyError error) {
102             // Handle the Volley error (e.g., log it)
103             Log.e("JsonParser", "Volley error", error);
104         }
105     });
106 }
107
108 public interface OnEventsParsedListener {
109     void onEventsParsed(List<Event> events);
110 }
111 }

```

3.3 Analisi di Clean Code

3.3.1 Nomi

I nomi di variabili risultano significativi, con gli attributi della classe che rispecchiano ciò che rappresentano, con alcune notevoli eccezioni:

- *root* per quanto abbastanza esplicativo, non rappresenta il root dell'URL ma bensì il path dell'API, *endpoint* o *path* sarebbero più esplicativi;
- *eventsList* riflette il contenuto della variabile, ma non è necessario specificare che si tratti di una lista, *events* sarebbe sufficiente;
- *eventObj* riflette il contenuto ma non specifica che si tratti di un oggetto JSON, *eventJson* potrebbe risultare più chiaro.

Per le funzioni, invece, i nomi risultano poco chiari e spesso non riflettono ciò che la funzione svolge realmente:

- *jsonParse()* non riflette il compito della funzione, che svolge molte altre operazioni oltre al semplice parsing di JSON come la creazione dell'URL, la gestione della connessione e infine il parsing del JSON e gestione degli errori.

Anche la classe risulta malamente nominata, *JsonParser* suggerisce una classe che esegue il parsing del JSON, in realtà questa si occupa anche di stabilire la connessione e gestirla.

3.3.2 Responsabilità

Come anticipato la funzione *jsonParse()* viola il **SRP (Single Responsibility Principle)** e il **CQS (Command Query Separation)**, occupandosi di diversi compiti quando potrebbe facilmente essere divisa in sotto funzioni:

- creare l'URL della API da chiamare;
- scrivere sul Log informazioni di debug;
- gestire la connessione HTTP verso la API;
- effettuare la chiamata alla API di TicketMaster;
- gestire la logica di parsing della risposta JSON;
- gestire la logica di eventuali errori di connessione o risposta;

Questi compiti dovrebbero essere estratti in funzioni separate, migliorando leggibilità e manutenzione del codice, riducendo contemporaneamente la complessità della funzione.

In generale tutta la classe dovrebbe modificata e divisa in due classi distinte:

- una che si occupi della gestione della connessione alla API di TicketMaster per recuperare i dati (gli eventi in questo caso);
- una che si occupi di analizzare e parsare questi dati in formato JSON, estraendo le informazioni richieste.

3.3.3 Commenti

Dal punto di vista dei commenti, la classe ne presenta un numero eccessivo, alcuni commenti sono utili e chiari, la maggior parte invece interrompe il flusso del codice spiegando concetti ovvi e non fornendo informazioni utili.

La spiegazione degli attributi della classe è superflua, il nome e il contesto dovrebbero essere sufficienti a comprendere cosa rappresentano.

In particolare la funzione *jsonParse()* presenta un commento generale che spiega in maniera poco chiara la sua funzionalità, e numerosi commenti all'interno che separano i blocchi di codice per funzionalità differenti, spesso spiegando banalità o fornendo informazioni poco utili.

Sono presenti anche commenti per codice ormai rimosso, come mostrato alla riga 40.

3.3.4 Formattazione

La formattazione del codice risulta mediamente buona, con la maggior parte delle righe che rientrano nei limiti consigliati, seppur con qualche eccezione legata principalmente a definizioni di costanti o stringhe hardcoded troppo lunghe.

Anche l'ordine delle funzioni risulta accettabile e logico; il problema si ha nella quantità di livelli di indentazione all'interno della funzione principale *jsonParse()*, facilmente risolvibile suddividendo questa in più funzioni di grandezza minore.

3.3.5 Hardocoding

La classe possiede un paio di problemi di embedding, in particolare la *chiave API* è hardcoded in chiaro come attributo statico della classe, ciò rappresenta un problema di sicurezza e manutenzione in caso di futuri cambiamenti.

Un'altra stringa che risulta essere hardcoded è l'URL base della API, che dovrebbe essere spostata in una costante o variabile globale così come la stringa *_embedded*, necessaria per eseguire il parsing della risposta JSON.

3.3.6 Gestione degli errori

La gestione degli errori è abbastanza approssimativa; spesso si cattura l'eccezione senza avere comportamenti condizionati al tipo di errore e non si comunica all'utente l'avvenuto problema, lasciandolo ad aspettare un evento che non si verificherà mai.

3.4 Code Refactoring

```
1 package it.marcosoft.ticketwave.NetworkActivity;
2
3 import android.content.Context;
4 import android.util.Log;
5
6 import com.android.volley.Request;
7 import com.android.volley.Response;
8 import com.android.volley.VolleyError;
9 import com.android.volley.toolbox.JsonObjectRequest;
10
11 import org.json.JSONArray;
12 import org.json.JSONObject;
13
14 import java.util.ArrayList;
15 import java.util.List;
16
17 import org.json.JSONException;
18
19 import it.marcosoft.ticketwave.EventModel.Event;
20
21 /**
22  * Utility class for parsing JSON responses from the Ticketmaster API.
23  */
24 public class TicketmasterClient {
25     private static final String TAG = "TicketmasterClient";
26
27     private static final String ENDPOINT_EVENTS = "discovery/v2/events";
28
29     private static final String API_KEY = "apikey=KqtxCDlnofSteZ63m7gmezFR8PR34o78";
30
31     private static final String API_URL = "https://app.ticketmaster.com/";
32
33     private static final String KEY_EMBEDDED = "_embedded";
34
35     private static final String KEY_EVENTS = "events";
36
37     private final String endpoint;
38
39     private final List<String> queryParams;
40
41     private final TicketMasterListener listener;
42
43     public TicketmasterClient(String endpoint, List<String> queryParams,
44         ↪ TicketMasterListener listener) {
45         this.endpoint = endpoint;
46         this.queryParams = queryParams;
47         this.listener = listener;
48     }
49
50     private String buildURL(){
```

```

50     StringBuilder urlBuilder = new
    ↳ StringBuilder(API_URL).append(endpoint).append("?");
51     for (String queryParam : queryParams) {
52         urlBuilder.append(queryParam).append("&");
53     }
54     return urlBuilder.append(API_KEY).toString();
55 }
56
57 private List<Event> fetchEventsFromJSON(JSONObject response) throws JSONException
    ↳ {
58     List<Event> events = new ArrayList<>();
59
60     if (!response.has(KEY_EMBEDDED)) {
61         return events;
62     }
63
64     JSONObject embedded = response.getJSONObject(KEY_EMBEDDED);
65     JSONArray eventsArray = embedded.getJSONArray(KEY_EVENTS);
66
67     for (int i = 0; i < eventsArray.length(); i++) {
68         JSONObject eventJson = eventsArray.getJSONObject(i);
69         events.add(new Event(eventJson));
70     }
71
72     return events;
73 }
74
75 private void notifyListener(List<Event> events) {
76     if (listener != null) {
77         listener.onEventsParsed(events);
78     }
79 }
80
81 private void handleResponse(JSONObject response) {
82     try {
83         if (ENDPOINT_EVENTS.equals(endpoint)) {
84             List<Event> parsedEvents = fetchEventsFromJSON(response);
85             notifyListener(parsedEvents);
86         } else {
87             String err = "Unsupported endpoint: " + endpoint;
88             Log.w(TAG, err);
89             if (listener != null) listener.onError(err);
90         }
91     } catch (JSONException e) {
92         String err = "JSON Parsing error";
93         Log.e(TAG, err, e);
94         if (listener != null) listener.onError(err);
95     }
96 }
97
98 public JsonObjectRequest createEventsRequest() {
99     String url = buildURL();
100     Log.d(TAG, "Request URL: " + url);
101
102     return new JsonObjectRequest(
103         Request.Method.GET,
104         url,
105         null,

```

```

106         response -> handleResponse(response),
107         error -> {
108             String err = "Volley error: " + error.getMessage();
109             Log.e(TAG, err);
110             if (listener != null) listener.onError(err);
111         }
112     );
113 }
114
115 public interface TicketMasterListener {
116     void onEventsParsed(List<Event> events);
117     void onError(String message);
118 }
119 }

```

3.5 Modifiche effettuate

Le modifiche apportate al codice originale sono diverse e mirate a risolvere i problemi citati nell'analisi di Clean Code.

La classe è stata rinominata in *TicketmasterClient*, per riflettere che interagisce con la API esterna di TicketMaster. Più ideale sarebbe la divisione della classe in due distinte, che si occupano di fare comunicazione con la API e parsing JSON in maniera separata, ma per semplicità di rappresentazione è stata mantenuta una singola classe.

Dal punto di vista dei nomi:

- l'attributo *root* è stato rinominato in *endpoint*, per rappresentare meglio il suo significato;
- l'attributo *eventsList* è stato rimosso, optando invece per una variabile locale *events* all'interno della funzione di parsing;
- la variabile *eventObj* è stata rinominata in *eventJson*, per specificare che si tratta di eventi JSON parsati;

La funzione *jsonParse()* è stata divisa in più funzioni con responsabilità distinte:

- *buildURL()* si occupa di creare l'URL della API da chiamare e ritornarlo;
- *fetchEventsFromJSON()* si occupa di eseguire il parsing del JSON e restituire la lista di eventi;
- *notifyListener()* si occupa di comunicare con l'oggetto Listener dopo che gli eventi sono stati ottenuti;
- *handleResponse()* si occupa di gestire la risposta della API, controllando eventuali errori e eseguendo la funzione di parsing;
- *createEventsRequest()* si occupa di creare la richiesta HTTP per ottenere gli eventi, utilizzando le funzioni precedenti per analizzare il JSON, notificare il listener e gestire la connessione.

Per risolvere i problemi di hardcoding invece ho apportato le seguenti modifiche:

- l'URL base della API è stata spostata in una costante statica *API_URL*, migliorando la manutenzione in caso di cambiamenti futuri;
- la chiave per il parsing del JSON è stata rimossa dal codice e spostata in una costante statica *KEY_EMBEDDED*, permettendo una facile modifica in caso di cambiamenti alla struttura del JSON;
- la chiave per ottenere gli eventi dal JSON è stata spostata in una costante statica *KEY_EVENTS*;;
- il tag per il Log è stato spostato in una costante statica *TAG*, così da poter essere modificato facilmente;
- il path per ottenere gli eventi è stato spostato in una costante statica *ENDPOINT_EVENTS*, permettendo di adattarsi ai cambiamenti futuri;

I commenti sono stati rimossi in quanto superflui, si ritiene che i nomi di variabili e funzioni siano autoesplicativi e non necessitino di commenti che ne approfondiscano il significato, come indicato anche dai principi di Clean Code.

4 Classe User.java

4.1 Ruolo

La classe svolge il ruolo di modello di dominio, andando a rappresentare un utente dell'applicazione, contenendo le sue proprietà principali quali: nome, cognome, mail e età.

Oltre a rappresentare l'utente, la classe utilizza l'interfaccia `Parcelable` di Android per permettere lo scambio di oggetti User in maniera efficiente tra le varie attività dell'applicazione.

4.2 Codice originale

```
1 package it.marcosoft.ticketwave.model;
2
3 import android.os.Parcel;
4 import android.os.Parcelable;
5
6 import com.google.firebase.database.Exclude;
7
8
9 public class User implements Parcelable {
10     private String name;
11     private String surname;
12     private int age;
13     private String email;
14
15     private String idToken;
16
17     public User(){
18
19     }
20
21     public User(String name, String email, String idToken) {
22         this.name = name;
23         this.email = email;
24         this.idToken = idToken;
25         this.surname="unknown";
26         this.age=0;
27     }
28
29
30     public User(String name, String email, String surname, int age, String idToken) {
31         this.name = name;
32         this.surname=surname;
33         this.age=age;
34         this.email = email;
35         this.idToken = idToken;
36     }
37
38     public String getName() {
39         return name;
40     }
41
42     public void setName(String name) {
```

```

43     this.name = name;
44 }
45 public String getSurname() {
46     return surname;
47 }
48
49 public void setSurname() {
50     this.surname = surname;
51 }
52
53 public int getAge() {
54     return age;
55 }
56
57 public void setAge() {
58     this.age = age;
59 }
60
61
62 public String getEmail() {
63     return email;
64 }
65
66 public void setEmail(String email) {
67     this.email = email;
68 }
69
70
71 @Exclude
72 public String getIdToken() {
73     return idToken;
74 }
75
76 public void setIdToken(String idToken) {
77     this.idToken = idToken;
78 }
79
80 @Override
81 public String toString() {
82     return "User{" +
83         "name='" + name + '\'' +
84         ", surname='" + surname + '\'' +
85         ", email='" + email + '\'' +
86         ", idToken='" + idToken + '\'' +
87         ", age='" + age + '\'' +
88         '}';
89 }
90
91 @Override
92 public int describeContents() {
93     return 0;
94 }
95
96 @Override
97 public void writeToParcel(Parcel dest, int flags) {
98     dest.writeString(this.name);
99     dest.writeString(this.email);
100    dest.writeString(this.idToken);

```

```

101     }
102
103     public void readFromParcel(Parcel source) {
104         this.name = source.readString();
105         this.email = source.readString();
106         this.idToken = source.readString();
107
108     }
109
110
111     protected User(Parcel in) {
112         this.name = in.readString();
113         this.email = in.readString();
114         this.idToken = in.readString();
115     }
116
117     public static final Creator<User> CREATOR = new Creator<User>() {
118         @Override
119         public User createFromParcel(Parcel source) {
120             return new User(source);
121         }
122
123         @Override
124         public User[] newArray(int size) {
125             return new User[size];
126         }
127     };
128 }

```

4.3 Analisi di Clean Code

4.3.1 Nomi

I nomi risultano significativi sia dal punto di vista di attributi e variabili sia dal lato delle funzioni. L'unico attributo mal nominato risulta essere `idToken` in quanto rappresenta una ambiguità e non è chiaro se si voglia rappresentare lo UserID o il TokenID del database di Firebase; nel primo caso andrebbe rinominato l'attributo, nel secondo andrebbe rimosso in quanto non pertinente con la classe `User`. Un errore più lieve si trova nel nome degli attributi `name` e `surname`, è comune utilizzare termini come `firstName` e `lastName` per evitare ambiguità in contesti di nomi con multipli termini.

La classe `User` rispecchia il concetto di utente, seppur con qualche ambiguità dovuta ancora una volta ad all'accoppiamento di diverse responsabilità, la classe dovrebbe rappresentare solo l'utente e le sue proprietà senza preoccuparsi di come scambiare i dati tra finestre di Android.

Altro problema di naming si trova invece nel metodo `describeContents()`, che risulta un nome esplicativo ma confusionario nell'ambito di utilizzo, in questo caso però risulta corretto in quanto parte della implementazione standard dell'interfaccia `Parcelable`.

4.3.2 Responsabilità

Come detto la classe viola il SRP (Single Responsibility Principle) in quanto dovrebbe limitarsi a rappresentare un utente e le sue proprietà, invece si occupa anche di gestire lo scambio di questi dati tra le differenti attività di Android.

Andrebbe divisa in due classi distinte, tra cui:

- una classe `User` che rappresenti l'utente e le sue proprietà (nome, email, data di nascita, ecc);
- una classe `UserSession` che si occupi dello scambio di informazioni tra le varie finestre e attività di android.

4.3.3 Formattazione

La formattazione risulta corretta, vengono rispettati i limiti consigliati di lunghezza orizzontale e grandezza delle funzioni; la classe non presenta annidamenti eccessivi, dovuti alla mancanza di funzioni molto complesse.

Il livello di astrazione viene rispettato e l'ordine delle funzioni segue una logica precisa, con setter e getter dello stesso attributo raggruppati insieme; le funzioni più complesse sono spostate verso il fondo della classe.

4.3.4 Hardcoding

La classe presenta un singolo problema di hardcoding, ovvero l'uso della stringa "unknown" per rappresentare un valore di `surname` non definito. Una soluzione migliore sarebbe utilizzare invece il valore `null`, che rappresenta meglio l'assenza di un valore, se lo si preferisce si potrebbe anche utilizzare una costante statica.

L'attributo `age` presenta un problema simile, con il valore iniziale settato a 0 a rappresentare un'età sconosciuta, questo però comporta ambiguità e problemi di logica. Sarebbe consigliato usare un altro valore, come -1, oppure cambiare tipo ad un `Integer` così da poter assegnare il valore `null`.

4.3.5 Commenti

Non sono presenti commenti nel codice della classe, in questo caso si tratta di un fattore positivo in quanto tutto il codice e i nomi delle variabili risultano autoesplicativi. Le funzioni sono inoltre brevi, semplice e chiare, non necessitano quindi di commenti aggiuntivi.

4.3.6 Errori logici

La variabile `age` è adatta ed esplicativa di cosa contiene, ma generalmente è preferibile utilizzare la data di nascita in rispetto all'età, per evitare il costante aggiornamento nella base dei dati.

Gli errori più gravi si trovano invece nella definizione di setter e getter, in particolare:

- il setter `setSurname()` non accetta un parametro di input, rendendolo inutile in quanto non è possibile modificare il `surname` con questo metodo;
- il getter `setAge()` ha lo stesso problema del metodo precedente, non accettando parametri in ingresso non risulta possibile modificare l'età dell'utente.

4.4 Code Refactoring

```
1 packbirthdate it.marcosoft.ticketwave.model;
2
3 import android.os.Parcel;
4 import android.os.Parcelable;
5 import java.util.Date;
6
7 import com.google.firebase.database.Exclude;
8
9
10 public class User implements Parcelable {
11     private String firstName;
12     private String lastName;
13     private Date birthDate;
14     private String email;
15
16     private String idToken;
17
18     public User(){
19
20     }
21
```



```

22 public User(String firstName, String email, String idToken) {
23     this.firstName = firstName;
24     this.email = email;
25     this.idToken = idToken;
26     this.lastName=null;
27     this.birthDate=null;
28
29 }
30
31 public User(String firstName, String email, String lastName, Date birthDate,
32 ↪ String idToken) {
33     this.firstName = firstName;
34     this.lastName=lastName;
35     this.birthDate=birthDate;
36     this.email = email;
37     this.idToken = idToken;
38 }
39
40 public String getfirstName() {
41     return firstName;
42 }
43
44 public void setfirstName(String firstName) {
45     this.firstName = firstName;
46 }
47
48 public String getlastName() {
49     return lastName;
50 }
51
52 public void setlastName(String lastName) {
53     this.lastName = lastName;
54 }
55
56 public Date getbirthdate() {
57     return birthDate;
58 }
59
60 public void setbirthdate(Date birthDate) {
61     this.birthDate = birthDate;
62 }
63
64 public String getEmail() {
65     return email;
66 }
67
68 public void setEmail(String email) {
69     this.email = email;
70 }
71
72 @Exclude
73 public String getIdToken() {
74     return idToken;
75 }
76
77 public void setIdToken(String idToken) {
78     this.idToken = idToken;

```

```

79     }
80
81     @Override
82     public String toString() {
83         return "User{" +
84             "firstName='" + firstName + '\'' +
85             ", lastName='" + (lastName != null ? lastName : "N/A") + '\'' +
86             ", email='" + email + '\'' +
87             ", idToken='" + idToken + '\'' +
88             ", birthDate='" + (birthDate != null ? birthDate.toString() : "N/A") +
89             "'}";
90     }
91
92     @Override
93     public int describeContents() {
94         return 0;
95     }
96
97     @Override
98     public void writeToParcel(Parcel dest, int flags) {
99         dest.writeString(this.firstName);
100        dest.writeString(this.email);
101        dest.writeString(this.idToken);
102    }
103
104    public void readFromParcel(Parcel source) {
105        this.firstName = source.readString();
106        this.email = source.readString();
107        this.idToken = source.readString();
108    }
109
110    }
111
112    protected User(Parcel in) {
113        this.firstName = in.readString();
114        this.email = in.readString();
115        this.idToken = in.readString();
116    }
117
118    public static final Creator<User> CREATOR = new Creator<User>() {
119        @Override
120        public User createFromParcel(Parcel source) {
121            return new User(source);
122        }
123
124        @Override
125        public User[] newArray(int size) {
126            return new User[size];
127        }
128    };
129 }

```

4.5 Modifiche effettuate

Partendo dai nomi, nel code refactoring sono state applicate le seguenti modifiche:

- l'attributo `name` è stato rinominato in `firstName` per maggiore chiarezza e standardizzazione;
- l'attributo `surname` è stato rinominato in `lastName` per maggiore chiarezza e standardizzazione;

- l'attributo `age` è stato rinominato in `birthDate` e il suo tipo cambiato da `int` a `Date`, per rappresentare la data di nascita invece che l'età, questo diminuisce la quantità di aggiornamenti necessari e permette di impostare un valore nullo in caso di data mancante;

La responsabilità della classe è rimasta invariata, rappresenta ancora un utente e le sue proprietà seppur contenendo anche la gestione delle activity legate ad Android.

Gli errori in setter e getter sono stati corretti, la classe offre ora la possibilità di modificare gli attributi `lastName` e `birthDate` tramite i rispettivi metodi `set`.

Il metodo `toString()` è stato modificato aggiungendo controlli per la data di nascita e per il cognome, in modo da non stampare `null` ma delle stringhe adatte, andando così a gestire il caso in cui questi siano mancanti o sconosciuti.